

如何调用 LLM 更准确地进行姓名消歧：工作总结报告

1. 研究目标

本项目研究的是 from-scratch author name disambiguation：给定同名或同拼音作者的一组论文，要求根据论文元信息判断哪些论文属于同一位真实作者。

传统方法通常依赖人工设计特征、图聚类或训练好的 embedding 模型。本项目尝试直接调用大语言模型，让 LLM 根据论文信息完成聚类。经过前期实验后，项目的问题不再被简单表述为“LLM 是否能做姓名消歧”，而是进一步拆成三个更具体的问题：

1. 应该给 LLM 哪些论文字段？
2. 应该如何组织这些论文信息？
3. prompt 应该如何写，才能让 LLM 更稳定地输出高质量聚类结果？

当前实验主要围绕 `title`、`abstract`、`keywords`、`coauthors`、`organization`、`venue` 等字段展开，并使用 Pairwise F1 与 B³ F1 评估聚类质量。

2. 传统数学与规则方法：WholsWho、BOND 与 MCCG

在探索 LLM 之前，我也学习了部分过去姓名消歧领域中更传统、更数学化的方法，主要包括 WholsWho toolkit、BOND 和 MCCG 三类规则消歧方法。这些方法的共同特点是：先把论文元信息转化为显式特征、相似度矩阵或图结构，再使用聚类算法或图神经网络得到作者簇。

比较两者方法，传统方法把姓名消歧形式化为可优化的数学问题，而 LLM 方法试图利用预训练模型已有的语义理解和推理能力，减少特征工程与训练成本。

2.1 WholsWho：手工特征、相似度矩阵与 DBSCAN 聚类

WholsWho toolkit 给出了较标准的姓名消歧流水线：数据加载、特征生成、模型构建和评估。在 SND 任务中，它的核心思路可以概括为：

1. 对同名作者 block 内的论文提取 semantic feature 和 relational feature；
2. 把每两篇论文之间的差异表示为距离矩阵；
3. 使用 DBSCAN 在预计算距离矩阵上聚类；
4. 对 DBSCAN 产生的 outlier 做 post-match 规则补救。

SND 部分：在 from-scratch name disambiguation (SND) 中，输入是一个同名作者 block 内的全部论文，没有候选作者 profile。WholsWho 的 SNDTrainer 会先计算 block 内任意两篇论文之间的 semantic distance 和 relational distance。若两类信息都使用，则合成距离为：

$$D(i, j) = \frac{D_{rel}(i, j) + w_{text} D_{sem}(i, j)}{1 + w_{text}}.$$

这个公式属于 SND 流程。其中：

- i, j 表示同一个姓名 block 内的两篇论文；
- $D(i, j)$ 表示论文 i 与论文 j 的最终综合距离，数值越小表示越相似；

- $D_{rel}(i, j)$ 表示 relational distance, 来自合作者、机构、venue 等关系特征;
- $D_{sem}(i, j)$ 表示 semantic distance, 来自标题、摘要等文本语义特征;
- w_{text} 是文本语义距离的权重, 用来控制 semantic feature 相对 relational feature 的影响;
- 分母 $1 + w_{text}$ 用于把加权和归一化, 避免只因为权重增加而整体放大距离。

随后, SND 流程使用 DBSCAN 对这个 block 内的距离矩阵聚类:

$$\hat{y} = \text{DBSCAN}(D, \epsilon, \text{min_samples}).$$

这个公式也属于 SND 流程。其中:

- D 是整个 block 的 $n \times n$ 论文距离矩阵, n 为该 block 中论文数量;
- ϵ 是 DBSCAN 的邻域半径阈值, 两篇论文距离不超过 ϵ 时可被视为密度邻居;
- min_samples 是形成核心点所需的最少邻居数量;
- $\hat{y}$ 是 DBSCAN 输出的预测簇标签向量, $\hat{y}_i$ 表示论文 i 被分到的簇;
- 若 $\hat{y}_i = -1$, 通常表示该论文被 DBSCAN 判定为 outlier。

DBSCAN 的直觉是: 如果一批论文在距离矩阵中彼此足够近, 就形成一个高密度区域; 密度不足的点会被标记为 outlier。它的优点是不需要预先指定真实作者数量, 适合 from-scratch 场景。

SND 中还会对 outlier 做 post-match。这里不再重新训练模型, 而是用合作者、机构、venue 和标题词的规则重叠, 把 DBSCAN 没有归入稳定簇的论文尝试并入已有簇。这个后处理逻辑和后面 BOND 的 outlier matching 很接近: 若某篇 outlier 与已有簇中论文的规则相似度达到阈值, 就合并; 否则保留为新的 singleton。

RND 部分: WhoIsWho toolkit 也支持 real-time name disambiguation (RND)。RND 与 SND 的输入形式不同: RND 要把一篇待分配论文与已有候选作者 profile 进行匹配。因此, RND 中的公式不是“block 内论文聚类”的公式, 而是“paper-profile 匹配特征”的公式。

在 RND 特征工程中, WhoIsWho 会对 coauthor、organization、venue、title、keywords 等字段计算:

- Jaro 字符串相似度;
- token set Jaccard 相似度;
- TF-IDF 加权的共同词比例;
- 合作者姓名稀有度加权;
- 候选作者 profile 中字段匹配的最大值和均值。

其中一类基础规则是 token set Jaccard。它既可以用于 RND 的 paper-profile 字段匹配, 也可以作为 SND / BOND 图构造中的关系相似度。对两个 token 集合 A, B :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

其中:

- A, B 表示两个待比较对象的 token 集合; 在 RND 中可以是一篇论文与候选作者 profile 的字段集合, 在 SND/BOND 中通常是两篇论文的字段集合;
- $A \cap B$ 表示两个集合的共同元素;
- $A \cup B$ 表示两个集合的全部不同元素;
- $|A \cap B|$ 和 $|A \cup B|$ 分别表示集合大小;

- $J(A, B)$ 越接近 1，说明两篇论文在该字段上越相似；越接近 0，说明重叠越少。

另一个更明显属于 RND 的规则是 paper-profile 的 TF-IDF 加权共同词比例。对某个字段的共同词，RND 中可以构造：

$$\text{ratio}_{\text{paper}} = \frac{\sum_{t \in W_p \cap W_a} \text{idf}(t) \text{tf}_p(t)}{\sum_{t \in W_p} \text{idf}(t) \text{tf}_p(t)}.$$

这个公式属于 RND 的候选作者匹配特征，而不是 SND 的 DBSCAN 聚类公式。其中：

- W_p 表示当前待分配论文在某个字段中的 token 集合；
- W_a 表示候选作者 profile 中同一字段的 token 集合；
- t 表示一个具体 token；
- $\text{tf}_p(t)$ 表示 token t 在当前论文字段中的出现次数；
- $\text{idf}(t)$ 表示 token t 的逆文档频率，越少见的 token 权重越高；
- 分子表示当前论文与候选作者 profile 共同出现 token 的加权得分；
- 分母表示当前论文该字段全部 token 的加权得分；
- $\text{ratio}_{\text{paper}}$ 越大，说明当前待分配论文在该字段上越接近候选作者 profile。

总结来说，WholsWho 的 SND 侧重点是“同名 block 内论文两两距离 + DBSCAN 聚类”，RND 侧重点是“待分配论文与候选作者 profile 的匹配特征 + 分类模型”。两者共享一些底层规则相似度思想，例如 coauthor 重叠、organization Jaccard、venue/title 词面重叠，但具体任务形式和公式不同。

这类规则方法的优势是清晰、可解释、可复现。它明确告诉模型：合作者重叠、机构重叠、venue 重叠和标题词重叠都应当影响论文是否属于同一作者。但它的问题也很明显：规则需要人工设计权重，字符串匹配很难理解深层语义，且跨领域迁移时需要重新调参。

2.2 BOND：规则图 + 图注意力网络 + 聚类自训练

BOND 的目标是 bootstrapping from-scratch name disambiguation。它比 WholsWho 的纯相似度聚类更进一步：先根据规则构图，再使用图神经网络学习论文表示。

它和 WholsWho SND 的一个关键区别是：WholsWho SND 基本是在人工构造的距离矩阵上运行一次 DBSCAN，然后用 post-match 处理 outlier；BOND 则把规则关系当作初始图结构，在训练过程中反复执行“GAT 表示学习 -> DBSCAN 产生伪聚类标签 -> 用伪标签继续优化 GAT”的循环。也就是说，BOND 中的 DBSCAN 不只是最后调用一次，而是在训练阶段承担动态生成 pseudo label 的角色。在经过一定次数迭代之后，使用当前的最新 pseudo label 进行 DBSCAN 聚类。

在 BOND 的图构造中，每篇论文是一个节点。若两篇论文共享作者、机构或 venue，就建立边，并记录边属性。例如边属性包括：

- 共同合作者数量；
- 共同机构 token 数量；
- organization Jaccard；
- 共同 venue token 数量；
- venue Jaccard。

若两篇论文 p_i, p_j 的合作者集合为 A_i, A_j ，机构 token 集合为 O_i, O_j ，则它们之间的关系强度可以用类似下面的规则打分：

$$s(i, j) = 1.5|A_i \cap A_j| + 1.0J(O_i, O_j) + 1.0J(V_i, V_j) + 0.33|T_i \cap T_j|.$$

其中：

- $s(i, j)$ 表示论文 i 与论文 j 的规则关系强度，数值越大表示越可能属于同一作者；
- A_i, A_j 表示两篇论文的合作者集合， $|A_i \cap A_j|$ 是共同合作者数量；
- O_i, O_j 表示两篇论文的机构 token 集合， $J(O_i, O_j)$ 是机构 Jaccard 相似度；
- V_i, V_j 表示两篇论文的 venue token 集合， $J(V_i, V_j)$ 是 venue Jaccard 相似度；
- T_i, T_j 表示两篇论文的标题 token 集合， $|T_i \cap T_j|$ 是共同标题词数量；
- 1.5, 1.0, 1.0, 0.33 是人工设定的字段权重，用来表达合作者、机构、venue 和标题词的重要性差异。

这里的权重体现了一个很强的领域假设：合作者重叠比标题词重叠更可靠。代码中的 post-match 也使用类似规则处理 outlier：如果某个 outlier 与已有非 outlier 节点的最高规则分数不低于阈值，就把它并入对应簇，否则新建 singleton 簇。

BOND 的模型部分使用 GAT。这里的 GAT 不是直接把初始边权 $s(i, j)$ 迭代更新成最终边权；更准确地说，初始规则边决定“哪些论文之间可以传递信息”，边属性提供关系强弱和类型信息，GAT 在这个图上学习每篇论文的表达向量。节点 i 从邻居 j 聚合信息时，不是简单平均，而是学习一个注意力权重 α_{ij} ：

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j \right).$$

其中：

- h_j 表示邻居节点 j 的当前表示向量；
- h'_i 表示节点 i 聚合邻居信息后的新表示向量；
- $\mathcal{N}(i)$ 表示节点 i 的邻居节点集合；
- W 是可学习的线性变换矩阵，用来把输入表示映射到新的特征空间；
- α_{ij} 是节点 i 对邻居 j 的注意力权重，表示聚合时应该多重视节点 j ；
- $\sigma(\cdot)$ 是非线性激活函数，例如 ELU。

注意力权重越大，说明模型认为节点 j 对节点 i 的表示越重要。这个过程会改变的是论文节点 embedding，而不是简单输出一张“更准确的新边权表”。训练完成后，BOND 再在 learned embedding 的 cosine distance 上运行 DBSCAN 得到预测簇。

训练时，BOND 会反复用当前 embedding 聚类得到 pseudo label，然后让模型预测论文对是否属于同一簇，并把模型预测得到的结果和 pseudo label 进行比较得到损失函数，再用损失函数优化 GAT embedding 模型。可以把它理解为一种自训练循环：

1. 使用当前 GAT embedding 计算论文之间的 cosine distance；
2. 在这个 embedding distance 上运行 DBSCAN，产生临时聚类标签；
3. 把临时聚类标签转成论文对矩阵 Y_{ij} ；
4. 模型根据 GAT 输出预测论文对相似性 $\hat{Y}_{ij}$ ；
5. 用二元交叉熵优化模型，使 embedding 更符合当前聚类结构；
6. 下一轮 epoch 再用更新后的 embedding 重新 DBSCAN，得到新的 pseudo label。

更具体地说，GAT 先为每篇论文输出一个节点表示 e_i 。代码中还会通过一个可学习的线性分类层，把节点表示变成一个 cluster-logit 向量：

$$q_i = e_i W_c + b_c.$$

其中：

- e_i 是论文 i 的 GAT embedding；
- W_c 是可学习的分类层权重矩阵；
- b_c 是可学习的偏置；
- q_i 是论文 i 的 cluster-logit 向量，不是最终真实簇标签，而是用于计算论文对相似性的中间表示。

然后，BOND 用两个节点向量的内积得到论文对的同簇打分：

$$r_{ij} = q_i^\top q_j, \quad \hat{Y}_{ij} = \sigma(r_{ij}).$$

其中：

- r_{ij} 是论文 i, j 的 pairwise logit；
- $\hat{Y}_{ij}$ 是经过 sigmoid 后的同簇概率；
- 若 q_i 和 q_j 方向相近， r_{ij} 更大， $\hat{Y}_{ij}$ 更接近 1；
- 若二者方向差异大， r_{ij} 更小， $\hat{Y}_{ij}$ 更接近 0。

DBSCAN 伪标签并不是一个“整个 cluster 的 attribute”直接压到模型上，而是先被转换成论文对标签矩阵：

$$Y_{ij} = \begin{cases} 1, & \text{论文 } i, j \text{ 在当前 DBSCAN 结果中属于同一簇;} \\ 0, & \text{论文 } i, j \text{ 在当前 DBSCAN 结果中属于不同簇.} \end{cases}$$

这样，每一篇论文 i 都会出现在许多 pairwise 约束中： $(i, 1), (i, 2), \dots, (i, n)$ 。因此，BCE 虽然写成对所有论文对求和，但会通过这些论文对约束落实到每一个节点表示上。

$$\mathcal{L}_{cluster} = - \sum_{i,j} \left[Y_{ij} \log \hat{Y}_{ij} + (1 - Y_{ij}) \log(1 - \hat{Y}_{ij}) \right].$$

其中：

- $\mathcal{L}_{cluster}$ 是聚类引导的 pairwise 二元交叉熵损失；
- i, j 遍历同一个 block 内的论文对；
- Y_{ij} 是由当前 DBSCAN pseudo label 得到的伪标签，若论文 i, j 属于同一伪簇则为 1，否则为 0；
- $\hat{Y}_{ij}$ 是模型预测论文 i, j 属于同一簇的概率或相似性得分；
- 第一项惩罚真实伪标签为同簇但模型预测过低的情况；
- 第二项惩罚真实伪标签为不同簇但模型预测过高的情况。

从梯度角度看，如果只看 q_i 这一端，它从所有与论文 i 相关的 pair 中接收误差信号，形式可以简化解为：

$$\frac{\partial \mathcal{L}_{cluster}}{\partial q_i} \approx \sum_j (\hat{Y}_{ij} - Y_{ij}) q_j.$$

其中：

- $\frac{\partial \mathcal{L}_{cluster}}{\partial q_i}$ 表示 cluster loss 对论文 i 中间表示 q_i 的梯度；
- $(\hat{Y}_{ij} - Y_{ij})$ 是论文对 (i, j) 的预测误差；
- q_j 是与论文 i 形成 pair 的另一篇论文的中间表示；
- $\sum_j$ 表示论文 i 会同时受到所有其他论文的 pairwise 约束影响。

这个公式说明了 BCE 如何落实到单个节点：如果 DBSCAN 伪标签认为 i, j 应该同簇，即 $Y_{ij} = 1$ ，但模型预测 $\hat{Y}_{ij}$ 偏低，那么梯度会推动 q_i 和 q_j 更接近；如果伪标签认为二者不同簇，即 $Y_{ij} = 0$ ，但模型预测偏高，那么梯度会推动它们分开。由于 q_i 来自 $e_i W_c + b_c$ ，而 e_i 又来自 GAT，误差会继续反向传播到 GAT 参数，进而改变下一轮得到的节点 embedding。

同时，图自编码器还会使用 reconstruction loss 保持图结构可重建。这个 reconstruction loss 的作用是防止模型只追随 DBSCAN 伪标签而完全忘记最初的规则图结构。整体损失可写作：

$$\mathcal{L} = \lambda \mathcal{L}_{cluster} + (1 - \lambda) \mathcal{L}_{recon}.$$

其中：

- $\mathcal{L}$ 是 BOND 的总训练损失；
- $\mathcal{L}_{cluster}$ 鼓励 embedding 支持当前聚类结构；
- $\mathcal{L}_{recon}$ 是图重构损失，鼓励 embedding 保留原始规则图中的连接结构；
- λ 是两类损失之间的权重，越大表示越强调 cluster-guided learning；
- $1 - \lambda$ 是 reconstruction loss 的权重。

因此，BOND 的逻辑可以概括为：规则构图提供初始关系骨架，GAT 在这个骨架上传播信息并学习论文表示，DBSCAN 反复在表示空间中给出伪聚类监督，与 GAT 原始预测模型对比，通过最小化二元交叉熵反向优化 GAT 的论文表示，最终再用 learned embedding 做聚类。它的优势是把规则关系、文本 embedding、图结构和聚类优化结合起来，比单纯 DBSCAN 更能学习复杂关系。但它仍然依赖预训练 paper embedding、图构造规则、DBSCAN 参数和自训练 pseudo label 的质量。如果初始图噪声很大，模型可能会继承甚至放大错误连接。

2.3 MCCG：多视图对比学习与聚类引导

MCCG 的出发点是：传统启发式图有噪声，单一视图容易不稳定，且 representation learning 与 clustering 往往没有被联合优化。因此它引入 multi-view contrastive learning 和 cluster-guided learning。

MCCG 同样把论文构造成图。每篇论文是节点，边来自 coauthor、organization、venue 等关系。假设某个同名 block 里有 n 篇论文，每篇论文的初始文本/关系 embedding 维度为 d ，那么原始图可以写成：

$$G = (X, A).$$

其中：

- $X \in \mathbb{R}^{n \times d}$ 是节点特征矩阵；
- 第 i 行 X_i 是论文 i 的初始特征向量，例如 paper text embedding；
- $A \in \{0, 1\}^{n \times n}$ 是邻接矩阵；
- $A_{ij} = 1$ 表示论文 i, j 之间有规则关系边，例如共享 coauthor、organization 或 venue；
- $A_{ij} = 0$ 表示两篇论文之间没有这类规则边；
- 这里的连接来自启发式规则，因此可能有噪声：有些真实同作者论文没有边，有些不同作者论文却因为共享机构或 venue 被连起来。

与 BOND 不同的是，MCCG 不只在一张固定图上训练，而是对同一个原始图生成两个增强视图：

$$G_1 = (X_1, A_1), \quad G_2 = (X_2, A_2).$$

其中：

- G_1, G_2 表示同一个原始论文图经过两次独立随机增强后得到的两个视图；
- $X_1, X_2 \in \mathbb{R}^{n \times d}$ 分别表示两个视图中的节点特征矩阵，shape 和原始 X 相同；
- $A_1, A_2 \in \{0, 1\}^{n \times n}$ 分别表示两个视图中的邻接矩阵，shape 和原始 A 相同；
- 两个视图保留相同的 n 个论文节点，但会随机遮蔽一部分特征维度和删除一部分边。

这里的“随机增强”指的是：对同一个原始图 $G = (X, A)$ 独立采样两组 dropout mask，并分别作用到节点特征和边上。dropout mask 可以理解为一个与原矩阵形状相同的 0/1 开关矩阵：值为 1 的位置保留原值，值为 0 的位置被删除或置零。它不是生成新论文，也不是改变 ground truth，而是在训练时构造两个带扰动的图输入，使模型不要过度依赖某一条具体启发式边或某一个特征维度。

$$A_1 = A \odot M_a^{(1)}, \quad A_2 = A \odot M_a^{(2)}.$$

其中：

- $M_a^{(1)}, M_a^{(2)} \in \{0, 1\}^{n \times n}$ 是两组独立采样的 edge dropout mask；
- $\odot$ 表示逐元素相乘；
- edge dropout mask 是作用在邻接矩阵 A 上的开关矩阵；
- 若 $M_{a,ij}^{(1)} = 0$ ，则原始边 A_{ij} 在第一个视图中被删除；
- 若 $M_{a,ij}^{(1)} = 1$ ，则原始边 A_{ij} 在第一个视图中保留；
- A_1, A_2 因为使用不同 mask，所以是同一图的两个不同扰动版本。

feature dropout 则发生在 X 上，即把一部分节点特征维度置零或削弱：

$$X_1 = X \odot M_x^{(1)}, \quad X_2 = X \odot M_x^{(2)}.$$

其中：

- $M_x^{(1)}, M_x^{(2)} \in \{0, 1\}^{n \times d}$ 是两个 dropout mask；
- 此处的 $\odot$ 同样表示逐元素相乘；
- feature dropout mask 是作用在节点特征矩阵 X 上的开关矩阵；
- 若 $M_{x,ik}^{(1)} = 0$ ，表示第一个视图中论文 i 的第 k 个特征被遮蔽；
- 若 $M_{x,ik}^{(1)} = 1$ ，表示该特征保留。

两个视图来自对节点特征和边的加权 dropout。所谓“加权”指的是每条边或每个特征维度被删除的概率不是完全相同，而是可以根据 degree、PageRank 或 eigenvector centrality 等图中心性调整。这样做的动机是：如果某个论文节点在不同扰动视图下仍然学到相近表示，那么它的表示更鲁棒；如果某条启发式边本身有噪声，模型也不会完全依赖它。

MCCG 使用 GAT 编码两个视图，得到：

$$z_1 = f_\theta(X_1, A_1), \quad z_2 = f_\theta(X_2, A_2).$$

其中：

- f_θ 表示参数为 θ 的 GAT encoder；
- $z_1, z_2 \in \mathbb{R}^{n \times h}$ 分别表示两种图视图下得到的论文节点 embedding；
- h 是 GAT 输出的 hidden dimension；
- 第 i 行 $z_{1,i}$ 是论文 i 在视图 G_1 下的表示；
- 第 i 行 $z_{2,i}$ 是同一篇论文 i 在视图 G_2 下的表示；
- X_1, A_1, X_2, A_2 与前一公式含义相同；
- 同一篇论文在两个视图中会得到两个表示，训练目标希望它们保持一致。

随后通过两个 projector 产生 multi-view representation 和 cluster representation。这里的 projector 不是新的图模型，也不是预先手工构造的规则，而是接在 GAT encoder 后面的可学习映射函数。它的参数在训练过程中和 GAT 参数一起通过反向传播学习得到。

所谓 `Linear -> activation -> Linear`，指的是一个两层 MLP。对输入表示 $u \in \mathbb{R}^h$ ，projector 的计算过程可以写成：

$$\text{projector}(u) = W_2 \phi(W_1 u + b_1) + b_2.$$

其中：

- u 是输入向量，例如某篇论文经过 GAT 后得到的 embedding；
- W_1, b_1 是第一层线性变换的权重矩阵和偏置；
- $W_1 u + b_1$ 是第一次 `Linear` 操作，本质是矩阵乘法加偏置；
- $\phi(\cdot)$ 是 activation，例如 ELU，用来引入非线性；
- W_2, b_2 是第二层线性变换的权重矩阵和偏置；
- $W_2 \phi(W_1 u + b_1) + b_2$ 是第二次 `Linear` 操作，得到 projector 的最终输出；
- W_1, b_1, W_2, b_2 都是训练中需要学习的参数。

如果输入是一整个矩阵 $Z \in \mathbb{R}^{n \times h}$ ，即 n 篇论文的 GAT embedding，那么 projector 会逐行作用到每篇论文上，输出一个新的表示矩阵：

$$\text{projector}(Z) = \phi(ZW_1 + B_1)W_2 + B_2.$$

这里 B_1, B_2 可以理解为把偏置复制到每一行后的矩阵形式。训练时，后面的 multi-view loss 或 cluster-guided loss 会产生误差信号，这些误差信号会反向传播到 projector 的 W_1, b_1, W_2, b_2 ，也会继续传回 GAT encoder。因此 projector 是被任务 loss 学出来的，而不是单独提前训练或人工设定的。

MCCG 中有两个 projector：

$$g_{mv}(\cdot) \quad \text{and} \quad g_c(\cdot).$$

其中：

- g_{mv} 是 multi-view projector，用于产生多视图对比学习使用的表示；
- g_c 是 cluster projector，用于产生聚类 and cluster-guided contrastive learning 使用的表示；
- 两个 projector 参数不同，因此即使输入来自同一个 GAT encoder，输出空间也不同。

具体计算可以写成：

$$H_{mv}^{(1)} = g_{mv}(z_1), \quad H_{mv}^{(2)} = g_{mv}(z_2),$$

$$z = \frac{z_1 + z_2}{2}, \quad H_{cluster} = g_c(z).$$

其中：

- $z_1, z_2 \in \mathbb{R}^{n \times h}$ 是 GAT 对两个增强视图输出的节点 embedding；
- $H_{mv}^{(1)}$ 是第一个视图的 multi-view representation；
- $H_{mv}^{(2)}$ 是第二个视图的 multi-view representation；
- $z = (z_1 + z_2)/2$ 表示把两个视图下的 GAT 表示做平均，得到一个融合后的节点表示；
- $H_{cluster}$ 是由融合表示 z 经过 cluster projector 后得到的聚类表示。

矩阵形状可以理解为：

$$H_{mv}^{(1)}, H_{mv}^{(2)} \in \mathbb{R}^{n \times d_{mv}}, \quad H_{cluster} \in \mathbb{R}^{n \times d_c}.$$

其中：

- $H_{mv}^{(1)}, H_{mv}^{(2)}$ 用于多视图对比学习；
- $H_{cluster}$ 用于 HDBSCAN 聚类和 cluster-guided contrastive learning；
- d_{mv} 是多视图 projector 输出维度；
- d_c 是 cluster projector 输出维度。

为什么要分成两个 projector？原因是两个训练目标并不完全相同。multi-view loss 关心“同一论文在两个扰动视图下是否稳定一致”，因此使用 $H_{mv}^{(1)}$ 和 $H_{mv}^{(2)}$ ；cluster-guided loss 关心“哪些论文应该在聚类空间中靠近”，因此使用 $H_{cluster}$ 。分开两个 projector 可以避免一个表示空间同时承担两种目标而互相干扰。

训练损失包含两部分。

第一部分是 multi-view contrastive loss。它鼓励同一论文在两个增强视图中的表示接近，而不同论文表示分开。为简化公式，下面用 $z_i^{(1)}$ 表示 $H_{mv}^{(1)}$ 的第 i 行，用 $z_i^{(2)}$ 表示 $H_{mv}^{(2)}$ 的第 i 行。温度系数为 τ 时，典型形式为：

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(z_i^{(1)}, z_i^{(2)})/\tau)}{\sum_k \exp(\text{sim}(z_i^{(1)}, z_k^{(2)})/\tau)}.$$

其中：

- $\mathcal{L}_i$ 是论文 i 的多视图对比学习损失；
- $z_i^{(1)}$ 表示论文 i 在第一个增强视图中经过 multi-view projector 后的表示，维度为 d_{mv} ；
- $z_i^{(2)}$ 表示论文 i 在第二个增强视图中经过 multi-view projector 后的表示，也是论文 i 的正样本；
- $z_k^{(2)}$ 表示第二个视图中论文 k 的表示，分母把所有候选论文作为对比对象；
- $\text{sim}(\cdot, \cdot)$ 通常表示 cosine similarity；
- τ 是温度系数，用来控制 softmax 分布的平滑程度；
- 该损失鼓励同一论文跨视图表示接近，同时让不同论文表示相对分开。

第二部分是 cluster-guided contrastive loss。MCCG 会用当前 cluster representation 运行 HDBSCAN 得到动态 cluster label，再把同一伪簇内的论文作为 positive pairs。为简化公式，下面的 z_i 表示 $H_{cluster}$ 的第 i 行。其 supervised contrastive 形式可以写成：

$$\mathcal{L}_i = -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{\sum_{a \neq i} \exp(\text{sim}(z_i, z_a)/\tau)}.$$

其中：

- $\mathcal{L}_i$ 是论文 i 的 cluster-guided contrastive loss；
- z_i 是论文 i 经过 cluster projector 后的 cluster representation，维度为 d_c ；
- $P(i)$ 是与论文 i 具有相同伪簇标签的正样本集合；
- p 表示一个正样本论文；
- a 表示除论文 i 之外的所有候选论文；
- $\text{sim}(z_i, z_p)$ 表示论文 i 与正样本 p 的相似度；
- $\text{sim}(z_i, z_a)$ 表示论文 i 与任意对比样本 a 的相似度；
- $|P(i)|$ 是正样本数量，用于对多个正样本取平均；
- τ 仍然是温度系数。

最终损失为：

$$\mathcal{L} = \lambda \mathcal{L}_{cluster} + (1 - \lambda) \mathcal{L}_{multiview}.$$

其中：

- $\mathcal{L}$ 是 MCCG 的总训练损失；
- $\mathcal{L}_{cluster}$ 是 cluster-guided contrastive loss，强调同伪簇论文表示接近；
- $\mathcal{L}_{multiview}$ 是多视图对比损失，强调同一论文在不同增强视图下表示稳定；
- λ 控制 cluster-guided loss 的权重；
- $1 - \lambda$ 控制 multi-view loss 的权重。

MCCG 的优势是更系统地处理图噪声：多视图扰动增强鲁棒性，cluster-guided loss 让 embedding 更贴近最终聚类目标，HDBSCAN 也比固定簇数方法更适合 from-scratch 场景。但它的代价是方法链条更复杂，需要图构造、数据增强、对比学习、HDBSCAN 参数和训练资源共同配合。

2.4 为什么仍然探索 LLM?

WhoIsWho、BOND 和 MCCG 说明，姓名消歧可以被非常清晰地数字化：论文变成节点，字段变成特征，字段重叠变成边，聚类变成优化目标。这些方法的优点是可控、可解释、适合大规模自动运行。

但它们也暴露出几个共同限制。

第一，特征工程成本高。传统方法需要人工决定哪些字段有用、如何归一化、如何计算相似度、不同字段如何加权。例如 BOND 中合作者、机构、venue、title 的规则权重需要人为设定；WhoIsWho 中大量字符串相似度和 TF-IDF 比例也依赖人工设计。

第二，语义理解有限。字符串相似度和 TF-IDF 能识别词面重叠，却很难理解“两个标题用不同术语描述相近研究问题”，也难以处理跨学科迁移、缩写、同义表达和上下文语义。

第三，错误会沿 pipeline 传播。图构造错了，GNN 会在错误边上传播信息；pseudo label 错了，自训练会强化错误；DBSCAN/HDBSCAN 参数不合适，会带来过拆分或过合并。

第四，方法依赖训练和数据准备。BOND 和 MCCG 需要 paper embedding、图数据、训练过程和 GPU 资源。对于一个想快速探索字段、prompt 或新数据集的研究流程，这个工程成本比较重。

LLM 方法的吸引力在于：它可以直接读取结构化论文信息，并利用预训练中获得的语言知识、学科知识和推理能力，把 title、coauthor、organization 等证据综合起来。我们不需要显式写出所有字符串规则，也不需要先训练一个图模型，就能得到较强的聚类结果。

当然，LLM 不是无成本的替代品。它存在 token 成本、输出不稳定、长 context 漏论文、过拆分和过合并等问题。因此在下文中，我们将探索如何把 LLM 调用变成一个可控的姓名消歧模块：

1. 用 T+O+C 控制输入证据和 token 成本；
2. 用 compact JSON 控制输出格式；
3. 用 full-coverage 校验控制生成错误；
4. 用调用前分流和调用后修正控制过拆分和过合并；
5. 在必要时结合传统图规则，对大 block 或异常 block 做局部补救。

因此，传统方法为我们提供了很好的数学参照：它们告诉我们哪些字段是身份线索、哪些相似度有用、哪些错误模式常见。而 LLM 方向的价值在于：尝试把这些线索交给具备语义理解能力的模型综合判断，从而减少手工规则和训练成本，并在复杂语义场景中获得更灵活的判断能力。

3. 实验设计

本项目的实验目标是要回答一个更实际的问题：如果希望把 LLM 作为一个可用的姓名消歧模块，应该怎样设计输入、组织上下文和编写 prompt，才能让它更准确、更稳定、更可控。

基于传统方法的学习，姓名消歧中真正有用的信息大致可以分成三类。第一类是论文内容线索，例如 title、abstract、keywords；第二类是作者关系线索，例如 coauthors；第三类是机构与发表环境线索，例如 organization、venue、year。传统规则方法需要手工把这些线索转成相似度、边权重或特征向量，而 LLM 有可能直接阅读这些结构化信息，并综合判断不同证据之间的关系。

因此，实验设计围绕三个逐步收窄的问题展开。

第一个问题是：应该给 LLM 哪些字段？

我的初始假设是，单一字段不足以稳定完成消歧，因为 title/abstract 偏向主题，coauthors 偏向关系，organization/venue 偏向身份约束；任何一种线索单独使用都可能导致过拆分或过合并。因此，第一组实验比较不同单字段和多字段组合，验证哪些字段组合既能提供足够身份信息，又不会带来过高 token 成本。这个实验先在 NA_Demo 三个作者 block 上做 pilot，再在 v3 validation 的 80 个作者 block 上验证 `title + coauthors + organization` (T+O+C) 的整体表现。

第二个问题是：在字段固定后，论文信息应该怎样组织？

当 T+O+C 被选为主要字段组合后，仍然存在一个不确定点：LLM 是否会受到论文排列方式影响。我的设想是，同样的字段内容，如果按不同顺序或中性分组方式提供，可能改变模型注意到的局部关系，从而影响聚类结果。因此，第二组实验固定 T+O+C，只改变 context organization，例如原始平铺、随机顺序、按年份排序、按 organization 分组、按 coauthor network 分组等，用来验证信息组织方式是否会带来性能变化或输出失败风险。

第三个问题是：在字段和论文顺序固定后，prompt 指令应该怎样写？

LLM 可能把姓名消歧误解成主题聚类，也可能过度相信某一个字段。因此，我的设想是，prompt 需要明确任务定义、解释不同证据的局限，并提醒常见错误。但过长、过复杂的 prompt 也可能增加输出不完整和 JSON 解析失败的风险。第三组实验固定作者、字段内容和论文顺序，只改变 prompt/context instruction，用来验证任务定义、证据

指导、错误提醒和分析步骤要求对聚类质量与输出稳定性的影响。

这三组实验对应的是同一个调用链条中的三个决策点：先决定给 LLM 什么信息，再决定这些信息如何排列，最后决定如何用 prompt 约束 LLM 的判断方式和输出格式。最终报告的结论也围绕这个逻辑组织：T+O+C 是当前最有性价比的默认字段组合；flat compact context 比复杂分组更稳定；prompt 应简洁明确地强调 author name disambiguation、证据互补和 over-splitting / over-merging 风险，同时必须配合严格的本地输出校验。

4. 评估指标

本项目主要使用两个聚类指标。

Pairwise F1

Pairwise 指标把同一作者论文对作为正例。若真实同作者的一对论文也被模型放在同一簇中，则是 true positive。Pairwise Precision 关注模型合并的论文对有多少是真的同作者；Pairwise Recall 关注真实同作者论文对有多少被模型成功合并。

$$P_{pair} = \frac{TP}{TP + FP}, \quad R_{pair} = \frac{TP}{TP + FN},$$

其中：

- TP 表示 true positive，即真实同作者、预测也同簇的论文对数量；
- FP 表示 false positive，即真实不同作者、但预测被合并到同一簇的论文对数量；
- FN 表示 false negative，即真实同作者、但预测被拆到不同簇的论文对数量；
- P_{pair} 是 Pairwise Precision，衡量模型合并出的论文对有多少是真的同作者；
- R_{pair} 是 Pairwise Recall，衡量真实同作者论文对有多少被模型成功合并。

$$F1_{pair} = \frac{2P_{pair}R_{pair}}{P_{pair} + R_{pair}}.$$

其中 $F1_{pair}$ 是 Pairwise Precision 和 Pairwise Recall 的调和平均值。若 precision 或 recall 任何一项很低，Pairwise F1 都会明显下降。

B³ F1

B³ 指标从单篇论文视角计算 precision 和 recall。对每篇论文，比较它所在预测簇与真实簇的重叠比例，再对所有论文求平均。B³ 对 singleton、过拆分和过合并都更敏感，也更适合观察作者 block 内的整体聚类质量。

对论文 i ：

$$P_i = \frac{|C_{pred}(i) \cap C_{true}(i)|}{|C_{pred}(i)|}, \quad R_i = \frac{|C_{pred}(i) \cap C_{true}(i)|}{|C_{true}(i)|}.$$

其中：

- i 表示一篇具体论文；
- $C_{pred}(i)$ 表示预测结果中包含论文 i 的簇；
- $C_{true}(i)$ 表示 ground truth 中包含论文 i 的真实作者簇；
- $C_{pred}(i) \cap C_{true}(i)$ 表示预测簇和真实簇的重叠论文集合；

- P_i 表示从论文 i 视角看，预测同簇论文中有多少确实属于同一真实作者；
- R_i 表示从论文 i 视角看，真实同作者论文中有多少被预测簇覆盖。

整体指标为：

$$P_{B^3} = \frac{1}{N} \sum_i P_i, \quad R_{B^3} = \frac{1}{N} \sum_i R_i,$$

其中：

- N 表示该作者 block 中论文总数；
- P_{B^3} 是所有论文级 precision 的平均值；
- R_{B^3} 是所有论文级 recall 的平均值；
- $\sum_i$ 表示对 block 内所有论文求和。

$$F1_{B^3} = \frac{2P_{B^3}R_{B^3}}{P_{B^3} + R_{B^3}}.$$

其中 $F1_{B^3}$ 是 B^3 Precision 和 B^3 Recall 的调和平均值，用来概括整个作者 block 的聚类质量。

在本项目中，Pairwise F1 更直观地反映论文对是否被正确合并； B^3 F1 更适合作为作者 block 级别的总体质量指标。

5. 实验结果

5.1 字段组合实验：T+O+C 是当前最合适的主组合

5.1.1 单字段效果不足

单独使用某一个字段时，LLM 聚类效果并不稳定。以 NA_Demo 三作者 pilot 为例：

字段	macro Pairwise F1	macro B^3 F1
coauthors	0.6654	0.7208
title	0.6140	0.6972
organization	0.6038	0.6804
abstract	0.5536	0.5666

单字段的问题在于证据类型过窄。`title` 和 `abstract` 主要提供研究主题信息，但同一作者可能跨主题，不同作者也可能研究相似主题；`coauthors` 是强身份线索，但合作网络可能变化，也可能缺失；`organization` 能提供机构约束，但作者会转机构，机构字符串也常有缺失或不同写法。

因此，单字段很难同时处理“同一作者跨主题/跨机构”和“不同作者同领域/同机构”的情况。

5.1.2 多字段组合显著优于单字段

在 NA_Demo 三作者 pilot 中，表现最好的字段组合集中在内容信息与身份信息互补的组合上：

字段组合	macro Pairwise F1	macro B ³ F1	complete author coverage
title+coauthors+organization	0.9792	0.9814	0.8773
abstract+coauthors+organization	0.9739	0.9766	0.8169
title+abstract+organization	0.9630	0.9587	0.7546
coauthors+organization	0.9158	0.9467	0.8645
title+organization	0.9277	0.9445	0.7546

其中 T+O+C 的 macro B³ F1 达到 0.9814，是三作者 pilot 中最强的组合。更重要的是，T+O+C 不依赖长摘要，token 成本相对低，且字段解释较清晰。

5.1.3 为什么是 title + coauthors + organization?

三类字段承担了不同功能。

`title` 提供研究主题信号。它可以帮助 LLM 识别同一真实作者长期研究的主题方向，也可以连接没有重复合作者但主题连续的论文。

`coauthors` 提供关系网络信号。重复合作者是强身份锚点，可以帮助修复同一作者被拆散的问题。pair-level 统计中，shared coauthor count 的同作者均值和不同作者均值差异非常大，说明合作者信息具有强区分能力。

`organization` 提供机构约束。它能帮助区分同名但不同机构的作者，也能在缺少合作者重叠时提供补充证据。pair-level 统计中，organization 的 same/different separation gap 高于 title 和 abstract。

因此，T+O+C 的优势不是“字段越多越好”，而是三类证据互补：主题、关系、机构分别覆盖了姓名消歧中的不同判断路径。

5.1.4 v3 80 作者验证结果

在 v3 validation 的 80 个作者 block 上，使用 T+O+C 的 LLM 粗聚类结果如下：

指标	mean	median	min	max
Pairwise F1	0.7852	0.8863	0.0000	1.0000
B ³ F1	0.8292	0.9076	0.0556	1.0000
Pairwise Precision	0.8586	0.9573	0.0000	1.0000
Pairwise Recall	0.7867	0.9056	0.0000	1.0000
B ³ Precision	0.9145	0.9545	0.4609	1.0000
B ³ Recall	0.8234	0.9142	0.0286	1.0000

80 个作者中，B³ F1 不低于 0.85 的有 55 个，B³ F1 低于 0.80 的有 21 个，B³ F1 低于 0.50 的有 6 个。

这说明 T+O+C 在多数作者上表现很强，但仍然存在失败模式，特别是大 block 或领域复杂 block 中的严重过拆分。例如 `bo_zou`、`yi_qian`、`jun_wang` 的 Pairwise F1 为 0，主要原因是模型几乎把每篇论文都分成 singleton，导致真实同作者论文对完全没有被合并。

5.2 信息组织实验：组织方式会影响结果，但稳定性风险很高

在字段组合实验后，T+O+C 被选为主要输入字段。但即使字段相同，LLM 看到论文的顺序和局部排列结构仍然可能影响判断。因此，信息组织实验要验证的问题是：**在输入字段完全相同的情况下，仅改变论文排列或中性分组方式，是否会改变 LLM 的姓名消歧结果？**

这个实验的核心控制变量是：

- 固定作者集合：选择 6 个作者，easy、medium、hard 各 2 个；
- 固定输入字段：每篇论文都只使用 title、coauthors、organization；
- 固定论文集合：同一个作者在所有 condition 下包含完全相同的 paper IDs；
- 固定字段内容：同一篇论文的 title、coauthors、organization 不允许变化；
- 固定模型和输出格式：仍使用相同 LLM、temperature 和 compact assignment JSON；
- 唯一改变的因素：论文在 prompt 中的顺序，或是否加入中性的 group 分隔。

也就是说，这组实验不是测试“更多字段是否更好”，也不是测试“prompt 指令是否更好”，而是专门测试 context organization 本身的影响。

具体设置如下。

`original_flat` 是 baseline。它保留原始数据中的论文顺序，把每篇论文平铺列出，不额外分组。这个条件代表最直接、最少加工的输入方式。

`random_order` 将同一作者 block 内的论文随机打乱。它的目的不是提升性能，而是测试 LLM 对输入顺序是否敏感。如果同一批论文只因为顺序不同就得到明显不同的聚类结果，说明 LLM 调用存在顺序不稳定性。

`chronological_order` 按 publication year 从早到晚排列论文。缺失年份的论文放在最后，同一年内按 paper ID 排序。这个设置的逻辑是：真实作者的研究主题、机构和合作者可能随时间变化，时间顺序可能帮助 LLM 看到一个作者 profile 的演化过程。

`organization_grouped` 按目标作者在该论文中的 organization 字符串进行分组。具体来说，先提取待消歧目标作者对应的 organization；再对 organization 做标准化，例如大小写归一、空白归一；organization 相同或标准化后相同的论文放在同一中性 group 中；缺失 organization 的论文放入缺失组。这里的 group 名称只使用 `Group 1`、`Group 2` 这类中性标签，不告诉 LLM “同组就是同一作者”。这个设置的逻辑是：organization 是强身份线索，把相同机构论文放近可能帮助 LLM 发现机构连续性；但也可能导致模型过度相信机构，从而忽略转机构或同机构同名作者。

`coauthor_network` 根据合作者关系构造论文之间的连接。具体来说，先从每篇论文的 authors 中去掉当前待消歧的目标姓名，只保留其他 coauthors；若两篇论文共享至少一个非目标 coauthor，就在这两篇论文之间建立连接；然后根据这些连接计算连通分量，同一连通分量中的论文放入同一中性 group；没有共享合作者的论文会成为单独 group。这个设置的逻辑是：重复合作者通常是很强的身份锚点，把 coauthor-connected papers 放近可能帮助 LLM 恢复同一真实作者的合作网络；但如果某些合作者跨多个真实作者，或合作网络很稀疏，它也可能引入错误合并或大量碎片。

`multifield_similarity` 不显示相似度分数，也不告诉 LLM 哪些论文相似，而是只用 title、organization、coauthors 在本地计算论文间相似度后重新排序。title 使用 TF-IDF cosine similarity，organization 使用 token Jaccard，coauthors 使用 coauthor set Jaccard；综合相似度取可用字段相似度的平均值。排序采用确定性 greedy 过程：从 paper ID 最小的论文开始，每一步选择与当前论文最相似的尚未排列论文放到后面。这个设置的逻辑是：如果相似论文在 prompt 中靠得更近，LLM 可能更容易看到局部连续性；但这种局部相似性也可能放大主题相似或机构相似带来的误导。

实验评估时，每次 LLM 输出都必须先通过 full-coverage 校验：所有输入 paper ID 必须恰好出现一次，不能缺失、重复或出现未知 ID。如果某次调用缺失论文或 JSON 无法解析，该次调用标记为 parse failure，不把它补成 singleton 后再参与主要均值计算。这样做的原因是：信息组织方式不仅会影响聚类质量，也会影响输出完整性；如果只统计成功调用的 F1，会高估复杂组织方式的实际可用性。

condition	mean B ³ F1	median B ³ F1	mean Pairwise F1	mean delta B ³ F1	parse failure
original_flat	0.8871	0.8848	0.8396	0.0000	0
random_order	0.8576	0.8788	0.7976	-0.0295	1
chronological_order	0.8507	0.7993	0.7879	-0.0194	1
organization_grouped	0.9020	0.9390	0.8436	0.0015	1
coauthor_network	0.8837	0.8873	0.8116	0.0059	2
multifield_similarity	0.8850	0.8846	0.8613	-0.0032	2

从成功解析的样本看，organization_grouped 的 mean B³ F1 最高，略高于原始平铺。但它也出现了解析失败和论文缺失问题，特别是在大 block 上更容易输出不完整。coauthor_network 在个别 hard case 上有收益，但 parse failure 更多，是高方差策略。

随机顺序平均降低表现，说明 LLM 对输入顺序并不完全鲁棒。对于同一组论文，仅改变排列方式就可能改变聚类结果。这一点对于大规模调用非常重要：如果实验不固定顺序，就很难判断性能变化来自字段、prompt，还是来自输入排列。

因此，当前更稳妥的组织方式不是复杂分组，而是：

- 1. 默认使用原始平铺或可复现的固定顺序；
- 2. 避免随机顺序；
- 3. 对大 block 谨慎使用 organization/coauthor 分组；
- 4. 如果使用分组，必须保留 full-coverage 校验，把缺失论文视为调用失败，而不是静默补 singleton。

5.3 Prompt 实验：简洁但有错误意识的 prompt 更有潜力

prompt/context instruction 实验固定作者、论文顺序和 T+O+C 字段，只改变任务说明。实验包含 3 个作者，条件包括：

condition	mean B ³ F1	median B ³ F1	mean Pairwise F1	parse failure
minimal	0.7150	0.7320	0.6805	0
task_definition	0.7816	0.7382	0.7321	0
evidence_guidance	0.7777	0.7777	0.7246	1
error_aware	0.8279	0.8279	0.8274	2
analyze_then_cluster	0.7338	0.7338	0.6837	2

`task_definition` 相比 `minimal` 明显提升 mean B³ F1，说明 LLM 需要明确知道这是 author name disambiguation，而不是普通主题聚类。仅仅说“把论文分组”容易让模型过度依赖主题相似性。

`error_aware` 在成功样本中 mean B³ F1 最高，说明提醒常见错误可以改变模型倾向。特别是在粗聚类任务中，应提醒模型：不要把同一真实作者因主题变化、机构变化或合作者变化而拆开；也不要完全缺少身份联系的不同作者粗暴合并。

但是 `error_aware` 和 `analyze_then_cluster` 都有更多失败调用，说明更复杂 prompt 也会提高输出不完整或格式失败风险。要求模型“先分析再聚类”并不一定更好，因为它可能消耗更多输出预算，或者在长 context 下更难保持完整 JSON。

因此推荐的 prompt 方向是：任务定义要清楚，证据解释要简洁，错误提醒要明确，但最终输出必须保持短、结构化、可校验。

6. 推荐的 LLM 调用流程

基于当前实验，推荐的 LLM 调用方式如下。

6.1 输入字段

默认使用：

```
title + coauthors + organization
```

不建议默认加入 abstract。虽然 abstract 在部分组合中有帮助，但它显著增加 token 成本，也可能引入更强的主题噪声。对于大规模粗聚类，T+O+C 是目前更好的性价比组合。

如果某个 block 的 title 很短、organization 缺失严重、coauthor 稀疏，可以考虑作为补救实验加入 abstract 或 keywords，但不应作为默认配置。

6.2 信息组织

默认使用稳定的 flat context，每篇论文一条记录：

```
record_index: 0
title: ...
organization: ...
coauthors: ...
```

推荐使用 `record_index -> cluster_id` 的 compact assignment 输出，而不是要求模型重复输出所有 paper id。这样可以减少输出 token，并降低 paper id 拼写错误风险。

不要随机打乱顺序。若需要重排，必须使用确定性规则，并在报告中记录规则。对于 `organization_grouped` 或 `coauthor_network` 分组，当前只能作为补充实验，不能直接替代 baseline。

6.3 Prompt 内容

推荐 prompt 包含四个部分。

第一，明确任务：

This is an author name disambiguation task. All records share the same normalized author name, but may belong to different real people.

第二，解释证据：

Titles provide topic evidence; organizations provide affiliation evidence; coauthors provide relationship evidence. None of them is individually decisive.

第三，提醒常见错误：

Avoid oversplitting one real author only because topic, organization, or collaborators change. Avoid overmerging different people only because one weak signal overlaps.

第四，强制输出格式：

Return JSON only. Every input record_index must appear exactly once in assignments.

不推荐让模型输出长篇解释。长解释可能帮助模型思考，但也增加截断、漏论文和 JSON 解析失败风险。更稳妥的做法是在 prompt 中要求模型内部综合判断，但最终只返回结构化结果。

6.4 调用参数

当前实验主要使用 `deepseek-v4-flash`、`temperature=0`。为了可复现，建议继续固定 temperature，并保存每次调用的原始 response、解析结果和 metadata。

每次调用前应估算 prompt token。对于超大 block，需要单独处理，不能简单把所有论文一次性塞给 LLM。当前失败案例表明，大 block 更容易出现过拆分、输出过长或缺失论文。

6.5 输出校验

LLM 输出必须经过本地校验：

1. 每个输入 record_index 必须出现；
2. 每个 record_index 只能出现一次；
3. 不允许未知 id；
4. clusters 不能为空；
5. 解析失败或缺失论文应标记为失败，而不是直接纳入指标均值。

对于实验报告，应同时记录 parse failure count。只看成功解析样本的 F1 会高估某些复杂 prompt 或复杂 context organization 的实际可用性。

7. 结果验证实验：推荐调用流程的局部修正效果

为了验证上面的推荐流程是否不只是来自总体均值，本节从 v3 validation 的 80 个作者 block 中挑选原始结果没有达到 0.95 的作者，使用推荐的 LLM 调用方式重新观察结果。这里的选择原则是：只展示成功调用、输出完整、且效果确实改善的作者；没有改善、解析失败或只有轻微波动的结果不放入主表，而放到后续失败模式和局限中讨论。

本节采用的验证设置为：

1. 字段仍然使用 `title + organization + coauthors`；

2. 信息组织采用确定性的 `original_flat` 形式，即每篇论文一条记录，不随机打乱，不额外分组；
3. 模型使用 `deepseek-v4-flash`，`temperature=0`；
4. 输出使用 `record_index -> cluster_id` 的 compact assignment；
5. 本地校验要求每篇论文恰好出现一次，不能缺失、重复或出现未知论文；
6. 指标使用与前文一致的 Pairwise F1 和 B³ F1。

对照组是 80 作者粗聚类分析中已有的 T+O+C 原始结果；验证组是后续按推荐流程重新组织上下文并调用 LLM 后得到的结果。

作者 block	论文 数	真实 作者 数	原始 Pairwise F1	验证 Pairwise F1	Pairwise 提升	原始 B ³ F1	验证 B ³ F1	B ³ 提升	原始 预测 簇数	验证 预测 簇数
zheng_hu	456	19	0.8663	0.9874	+0.1211	0.9299	0.9913	+0.0614	24	22
yi_qian	651	19	0.0000	0.9647	+0.9647	0.0567	0.9389	+0.8822	651	15

这两个案例说明，LLM 的消歧效果并不只取决于字段本身，也强烈依赖调用方式是否稳定。`zheng_hu` 的原始结果已经不算失败，但仍然存在过拆分；使用确定性的 flat context 和更明确的任务约束后，Pairwise F1 从 0.8663 提升到 0.9874，B³ F1 从 0.9299 提升到 0.9913，说明模型能够更好地把同一真实作者的论文保持在同一簇内。

`yi_qian` 的原始结果更极端：原始预测几乎退化为每篇论文一个 singleton，因此 Pairwise F1 为 0。重新调用后，预测簇数从 651 降到 15，Pairwise F1 提升到 0.9647，B³ F1 提升到 0.9389。这说明对于部分看似“LLM 完全失败”的 block，失败并不一定来自字段无效，也可能来自一次调用中的上下文组织、输出预算或模型决策不稳定。稳定的 flat context、compact assignment 和严格校验可以把一部分异常结果恢复到可用水平。

不过，这个验证实验不能被解释为“推荐流程一定能修复所有低分作者”。在补充测试中，也存在重新调用后没有提升、甚至略有下降的作者。因此，本节更适合作为局部正例：它证明推荐流程有实际修正能力，但是否需要重跑、何时重跑、是否加入二阶段分块，仍然需要依赖后续的失败检测策略。

8. 主要失败模式

当前最重要的失败模式是 over-splitting。某些作者 block 被拆成大量 singleton，导致 Pairwise Recall 接近 0。`bo_zou`、`yi_qian`、`jun_wang` 在原始 80 作者结果中是典型例子。第 7 节中的 `yi_qian` 说明，这类失败中有一部分可以通过稳定调用方式恢复，但这并不意味着所有 singleton 失败都能被简单重跑修复。

这类失败并不是 JSON 格式问题，也不是本地 fallback 造成的，而是 LLM 真实返回的聚类倾向。可能原因包括：

1. block 太大，模型难以维持全局一致性；
2. 论文主题跨度大，模型过度依赖 title 主题差异；
3. 化学、材料、生物医学等领域标题包含大量缩写、公式或实体名，LLM 可能难以稳定判断连续性；
4. coauthor 和 organization 虽然存在，但模型没有足够重视关系连续性；
5. prompt 对“同一作者可能跨主题/跨机构/跨合作者”的强调仍然不够稳定。

第二类失败是 over-merging。模型把多个真实作者合并到一个大簇中，通常会损害 Pairwise Precision。这在整个消歧过程前期的粗分类目标下可以部分接受，因为粗分类更强调 recall，但如果混入过多不同作者，后续的簇内精分压力会变大。

第三类失败是输出完整性失败，包括缺失论文、重复分配、跨 cluster 重复、JSON 不完整或 API incomplete read。这类失败与聚类质量不同，应单独统计。

9. 后续研究方向：在 LLM 调用前后加入判断与修正

当前实验说明，LLM 并不是一个只要输入论文信息就能稳定得到最优聚类的黑箱。它在很多作者 block 上表现很好，但也会出现过分、过合并、输出缺失和格式不稳定。因此，更有研究价值的方向不是继续无条件增加字段或扩大 prompt，而是研究如何在 LLM 调用前后加入判断、分流、预处理和修正机制。

9.1 调用前：判断哪些 block 适合直接交给 LLM

不同作者 block 的难度差异很大。有些 block 的 coauthor 和 organization 信息非常清晰，T+O+C 就足以让 LLM 得到很高 F1，甚至成本更低的本地规则分类也能获得较好的效果；有些 block 论文数量过多、机构缺失严重、标题主题跨度大，直接一次性调用 LLM 容易产生过拆分或输出不完整。

因此，后续可以研究一种调用前的可靠性判断机制：在不调用 LLM 的情况下，先根据 paper count、coauthor 连通性、organization 覆盖率、title 主题分散度、候选块规模等特征，预测一个作者 block 是否适合直接使用 T+O+C flat prompt。对于高可靠 block，可以直接调用 LLM；对于低可靠 block，则先进行分块、补充字段或采用更谨慎的 prompt。

目前，已经初步尝试过 router 实验，但当前 80 作者数据较小，复杂模型还没有稳定证明有效。因此，后续更合理的方向是考虑先总结 LLM 聚类成功与失败的边界特征，再建立更可解释的规则或轻量模型。

9.2 调用前：对输入数据做确定性预处理

LLM 的输入顺序和组织方式会影响结果，但随机重排会引入不可复现性。因此，更值得研究的是确定性的预处理方式。例如，可以先根据共同合作者、目标作者机构、论文年份或标题相似度，对论文进行排序或局部分组，再把这些结构化信息交给 LLM。

这种预处理的目标不是替代 LLM 聚类，而是降低 LLM 需要在长上下文中自行发现结构的难度。特别是对于大 block，可以先用 coauthor/organization 图得到若干候选子块，再让 LLM 判断子块内部是否应继续拆分、子块之间是否需要合并。这样可以把一次超长、不稳定的聚类任务，转化为多个更小、更可控的判断任务。

9.3 调用后：判断 LLM 结果是否可信，并进行修正

LLM 输出后不能只检查 JSON 是否可解析，还应该检查聚类结果本身是否可信。可能的后验信号包括：预测簇数是否异常大、singleton 比例是否异常高、最大簇是否接近 catch-all、同一簇内 organization/coauthor 是否完全断裂、不同簇之间是否存在大量共同合作者或高度相似机构。

如果这些信号显示结果不可靠，可以触发后续修正流程。例如，对于 singleton 过多的结果，可以让 LLM 专门判断这些 singleton 是否应并入已有大簇；对于过大的混合簇，可以在簇内加入 abstract、keywords、venue、year 等更细字段，让 LLM 重新判断是否需要拆分；对于缺失或重复论文，则直接标记为调用失败并重跑。

也就是说，后续系统不应该把 LLM 的第一次输出视为最终答案，而应该把它看作一个需要被验证的候选聚类结果。真正有研究价值的问题是：如何定义 LLM 聚类结果的可靠性，如何发现错误边界，以及如何用最小的额外调用成本修正这些错误。

10. 总体结论

当前最可靠的 LLM 姓名消歧调用策略可以概括为：

1. 默认粗聚类输入 `title + coauthors + organization`；
2. 使用固定、可复现的 flat context，不随机打乱论文；
3. prompt 要明确这是 author name disambiguation，而不是主题聚类；

4. prompt 要简洁解释 title/coauthor/organization 的互补作用；
5. prompt 要提醒 over-splitting 和 over-merging 两类错误；
6. 最终输出必须是短 JSON，并要求每篇论文恰好出现一次；
7. 本地必须做 full-coverage 校验，把解析失败和缺失论文单独记录；
8. 对超大 block 和异常 singleton 结果，不能盲信一次 LLM 输出，应使用分块或二次检查。

整体而言，LLM 做姓名消歧不是简单地“给越多信息越好”，而是要给它最稳定、最身份相关、token 成本可控的证据，并用严格的输出格式和本地校验把不可控生成压回可评估流程中。当前实验表明，`title + coauthors + organization` 是最适合作为默认粗聚类配置的字段组合；复杂 context organization 和更长 prompt 可能带来局部收益，但也会明显增加输出失败和不稳定风险。

11. 当前证据的局限

本报告的结论仍然有几个边界，需要在后续工作中进一步验证。

首先，LLM 姓名消歧效果会受到模型版本、上下文长度、输出格式约束和 API 行为的影响。不同模型对 title、coauthor、organization 的理解能力不完全相同，因此本报告得到的是当前模型条件下较稳妥的调用策略，而不是与模型无关的绝对最优规则。

其次，姓名消歧任务本身具有明显的分布差异。不同学科、不同语言姓名、不同机构书写习惯、不同合作者密度都会改变任务难度。T+O+C 在当前数据上表现出较好的性价比，但在更极端的缺失字段、跨领域迁移或超大作者 block 中，仍可能需要额外的预处理和修正机制。

第三，LLM 方法的准确率和工程可用性不能完全分开。即使聚类逻辑本身有效，长上下文调用仍可能带来输出截断、论文遗漏、格式错误和成本波动。因此，一个真正可用的系统必须同时考虑指标表现、token 成本、输出稳定性和失败恢复。

第四，离线指标只能近似反映真实应用目标。Pairwise F1 和 B³ F1 能衡量聚类质量，但它们不能完全表达研究者在实际使用中对可解释性、可人工检查性和错误修正成本的需求。因此，后续仍需要把自动指标和人工可读的错误分析结合起来。

因此，当前最适合写入最终方法总结的表述是：T+O+C flat compact prompt 是当前证据下最稳妥、最高性价比的默认调用方案；更复杂的上下文组织和 prompt 设计应作为针对异常 block 的补充策略，而不是默认替代方案。

12. 参考文献

- [1] Chen et al. Web-Scale Academic Name Disambiguation: The WhoIsWho Benchmark, Leaderboard, and Toolkit. 2023.
- [2] Cheng et al. BOND: Bootstrapping From-Scratch Name Disambiguation with Multi-task Promoting. 2024.
- [3] Ye F., Xia Z., Ling Z., Wu L. Multi-view contrastive and cluster-guided learning for author name disambiguation. Expert Systems with Applications, 289:128324, 2025. DOI: 10.1016/j.eswa.2025.128324.